

ISPTEC
Instituto Superior Politécnico de Tecnologias e Ciências

ESTRUTURA DE DADOS I

Autor: Sílvia António
Área: DET

O QUE VAMOS ESTUDAR ?

I. Revisão da linguagem C
II. Ponteiros e Tipos Abstractos de Dados
III. Listas
IV. Pilhas
V. Filas
VI. Arvores

Autor: Sílvia António
Ano Académico: 2026/2027

Ponteiros e Tipos Abstractos de Dados

- Ponteiros
- **Alocação Dinâmica de memória**
- Tipos Abstractos de Dados

Autor: Sílvia António
Ano Académico: 2026/2027

Alocação dinâmica

- Quando declaramos um vector, atribuímos um tamanho fixo que não pode ser alterado.
- Alguns problemas, necessitam de uma estrutura mais flexível, onde o tamanho possa ser ajustado de acordo com a necessidade.

Autor: Sílvia António
Ano Académico: 2026/2027

Alocação dinâmica

- É a alocação de memória feita de forma manual, durante a execução do programa.
- A alocação é feita por meio das funções: **malloc**, **calloc**, **realloc** e **free**. Estas funções estão definidas na biblioteca **stdlib.h**.



Autor: Sílvia António
Ano Académico: 2026/2027

Funções de alocação dinâmica

- **malloc –** reserva um conjunto de bytes indicados. Devolve um ponteiro para o conjunto criado ou NULL caso a alocação falhe.

- Sintaxe: **void * malloc(size);**

Exemplo:

```
float * ptr = NULL;
ptr = (float*) malloc(100 * sizeof(float)); //reserva 400 bytes
```

Funções de alocação dinâmica

- **calloc –** reserva memória para um conjunto de n elementos do tamanho indicado. Devolve um ponteiro para o conjunto criado ou NULL caso a alocação falhe.

- Sintaxe: **void * calloc(n, size);**

Exemplo:

```
float * ptr = NULL;
ptr = (float*) calloc(25, sizeof(float)); //reserva espaço contíguo
para                                                    //25 variáveis
```

Funções de alocação dinâmica

- **realloc –** permite alterar o número de bytes reservado pelo malloc ou calloc. Devolve um ponteiro para o conjunto criado ou NULL caso a alocação falhe.

- Sintaxe: **void * realloc(ptr, newSize);**

Exemplo:

```
float * ptr = NULL;
ptr = (float*) malloc(25 * sizeof(float)); //reserva espaço
ptr = (float*) realloc(ptr, 30 * sizeof(float));
```

Funções de alocação dinâmica

- **free –** libera a memória reservada pelas funções de alocação.
- Sintaxe: **free (ptr);**



Autor: Sílvia António
Ano Académico: 2026/2027

AULA 11_12 – Ponteiros - Alocação - Exemplo


```c
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n, i, *ptr;

 printf("Insira a quantidade de números: ");
 scanf("%d", &n);

1  ptr = (int*) malloc(n * sizeof(int));

 if(ptr == NULL) {
2   printf("Erro de alocação!");
    exit(0);   }

  printf("Insira %d números: ", n);
3 for(i = 0; i < n; i++) {
    scanf("%d", ptr + i);  }

  for(i = 0; i < n; i++)
4      print("%d,  ", *(ptr + I));

5   free(ptr);   // liberar a memória alocada


  return 0;
}
```

1- Reserva memória para guardar N números inteiros de forma manual, usando a função malloc. Retorna o endereço da memória reservada por meio de um ponteiro.
2- Se a função retornar NULL para o ponteiro, é porque não há espaço disponível.
3 - Lê os números inseridos pelo utilizador e os guarda na memória, por meio de ponteiros.
4- Lê os números guardados na memória por meio de ponteiros e os imprime na tela.
5 – Libera a memória usada pelos números.

Exercícios

1. Implemente a função **char * Repete(char * str, int n)** que cria dinamicamente uma nova string com n cópias da string original, separadas por espaços, excepto a última ocorrência.

   Ex: **String original** "ana", n =3; **Resultado**: "ana ana ana", sem as aspas.

2. Lê um valor **n**, reserva dinamicamente um array de **n** inteiros, preenche-o com números lidos do teclado, calcula a média e liberta a memória.

Autor: Sílvia António
Ano Académico: 2026/2027

Resolução de exercícios - Lista4

Autor: Sílvia António
Ano Académico: 2026/2027

BIBLIOGRAFIA

1. Damas. Luis, (2015), Linguagem C, 24ª Ed, FCA
2. Sedgewick, R. & Wayne, K. (2011). Algorithms in C, 3ª Ed.



MUITO OBRIGADO PELA ATENÇÃO!